Федеральное государственное автономное образовательное учреждение высшего образования

Национальный исследовательский ядерный университет «МИФИ»

Кафедра «Криптология и кибербезопасность»

Запечников С.В.

Методические указания по выполнению лабораторного практикума «Программная реализация криптографических протоколов и схем»

по курсу «Прикладная криптография»

Москва - 2026

Содержание

Введение……3

Часть 1. Обеспечение безопасности данных при их передаче и хранении……...4

Лабораторная работа №1. Реализация модели инфраструктуры открытых ключей...……5

Лабораторная работа №2. Реализация модели менеджера паролей……11

Лабораторная работа №3. Реализация модели системы защищенного распределенного хранения данных……16

Лабораторная работа №4. Реализация модели защищенного канала передачи данных согласно спецификации TLS 1.3……22

Введение

Лабораторный практикум «Программная реализация криптографических протоколов и схем» состоит из двух частей, выполняемых в ходе изучения первой и второй половин двухсеместрового курса «Прикладная криптография».

Цель практикума заключается в приобретении навыков разработки и реализации несложных средств и систем криптографической защиты информации (а точнее, их учебных моделей) в соответствии с техническими заданиями (ТЗ), которыми по сути своей являются приводимые в этих методических указаниях описания лабораторных работ. Практикум позволяет познакомиться с устройством довольно широкого круга современных средств криптографической защиты информации, функционирующих в составе системного и прикладного ПО.

Предполагается, что средства разработки (языки программирования, библиотеки, среды разработки и отладки программ) выбираются студентами самостоятельно. Никаких ограничений в выборе нет. Процесс взаимодействия между сторонами протокола может быть реализован как с применением сетевых технологий, так и при помощи буферных переменных, моделируя участников объектами. Главное – это реализовать требуемую в ТЗ функциональность. Все решения, касающиеся деталей реализации, не описанных в методических указаниях, следует принимать самостоятельно. Везде, где это возможно, при реализации следует использовать криптографические примитивы, описанные в российских (межгосударственных) ГОСТах по криптографической защите информации (или в рекомендациях по стандартизации).

Последовательно выполняемые лабораторные работы предполагают некоторую зависимость от ранее выполненных работа. Так, результаты лабораторной работы №1 - «Реализация модели инфраструктуры открытых ключей» можно и нужно использовать при выполнении следующих лабораторных работ, в частности, работ №№3 – 6, а результаты лабораторной работы №4 – при выполнении работы №5.

Лабораторные работы №№6,7 отличаются повышенной сложностью.

Автор желает учащимся успешной работы над лабораторным практикумом!

Часть 1.

Обеспечение безопасности данных при их передаче и хранении

Лабораторная работа №1. Реализация модели инфраструктуры открытых ключей

1. Инфраструктура открытых ключей

Пусть имеется криптосистема, включающая большое число участников (рис. 1). Среди участников криптосистемы выделяется один, специальный, которому доверяют все остальные. Он получает название *удостоверяющий центр*, или *центр сертификации ключей*, или *агентство сертификации* (*Certification Authority – CA*). Его функции может выполнять, например, администратор системы, оснащенный соответствующим аппаратным и программным обеспечением (сервер регистрации и сертификации ключей). Все остальные участники считаются обычными, «рядовыми» клиентами системы.

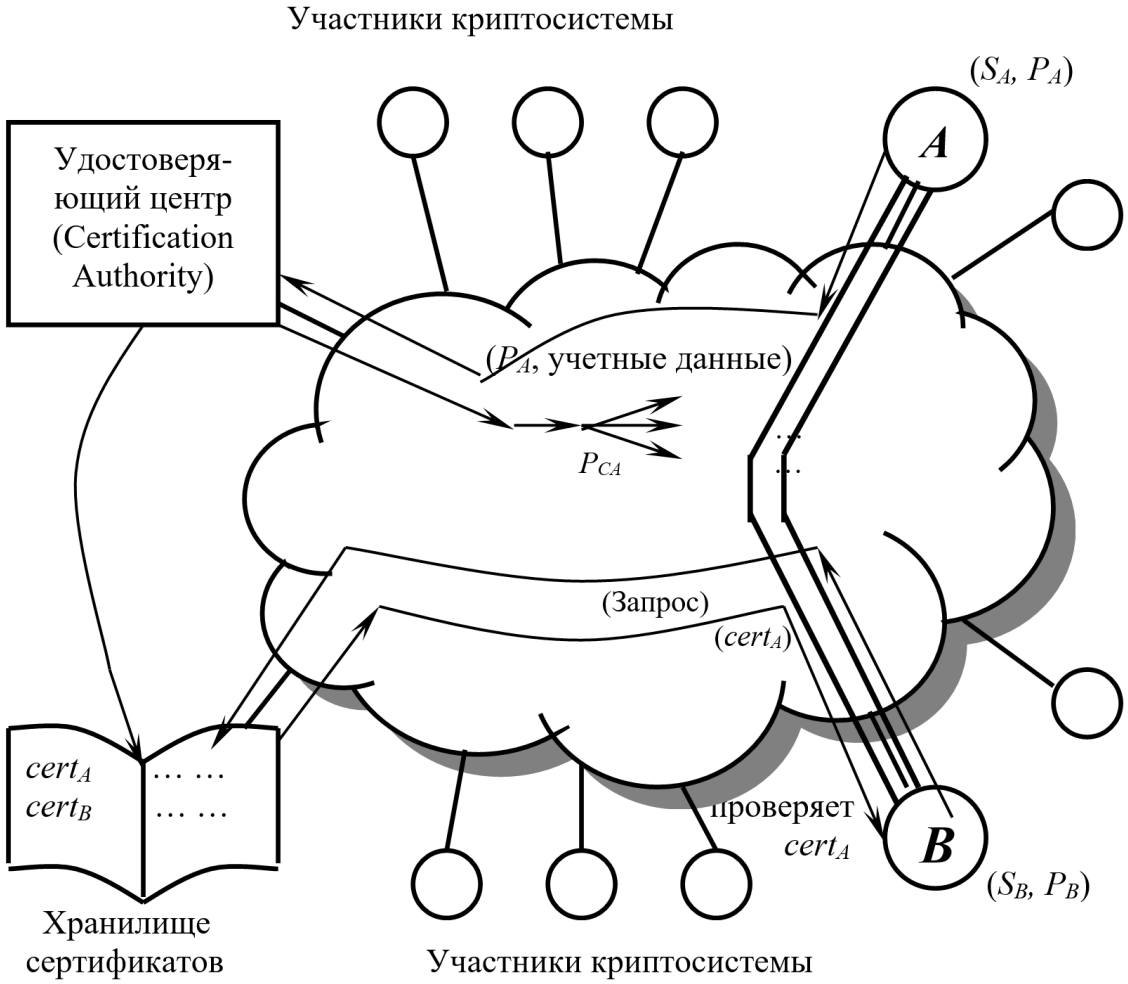

Рис. 1. Процессы получения и использования сертификатов участниками криптосистемы

При введении в систему каждого из этих участников – возьмем для примера участника *A* – он должен пройти процедуру регистрации в криптосистеме (что соответствует первой фазе жизненного цикла его криптографического ключа). Для этого он контактирует с удостоверяющим центром, чтобы зарегистрировать свой открытый ключ и получить от него так называемый *сертификат* своего открытого ключа. Удостоверяющий центр должен проверить представленные ему учетные данные, а также (что очень важно!) знание секретного ключа, соответствующего представленному для регистрации открытому

ключу. Решить эту задачу можно различными способами: в самом простом случае удостоверяющий центр может попросить *A* зашифровать на своем секретном ключе текст заданного формата и проверить правильность его расшифрования при помощи представленного открытого ключа, а можно воспользоваться протоколом доказательства знания с нулевым разглашением знания.

Сертификат открытого ключа – электронный документ в форме специальной структуры данных, состоящей из полей данных и поля подписи. Поля данных содержит атрибуты участника и его открытый ключ, а также атрибуты удостоверяющего центра. Поле подписи содержит цифровую подпись удостоверяющего центра под полем данных, криптографически связывающую признаки участника с его открытым ключом.

Все участники криптосистемы, заинтересованные в связи с участником *A*, получают впоследствии его сертификат либо путем прямого запроса сертификата у участника *A*, либо извлекая его из открытого общедоступного справочника, который заводится в криптосистеме. Сертификат, таким образом, является средством для хранения, распространения и передачи через небезопасные каналы связи открытых ключей без опасения их необнаруживаемого изменения.

Основным стандартом по сертификатам является стандарт Международного Союза электросвязи ITU X.509. В соответствии со стандартом X.509 сертификат имеет формат, показанный на рис. 2 (в разных версиях стандарта он немного различается).

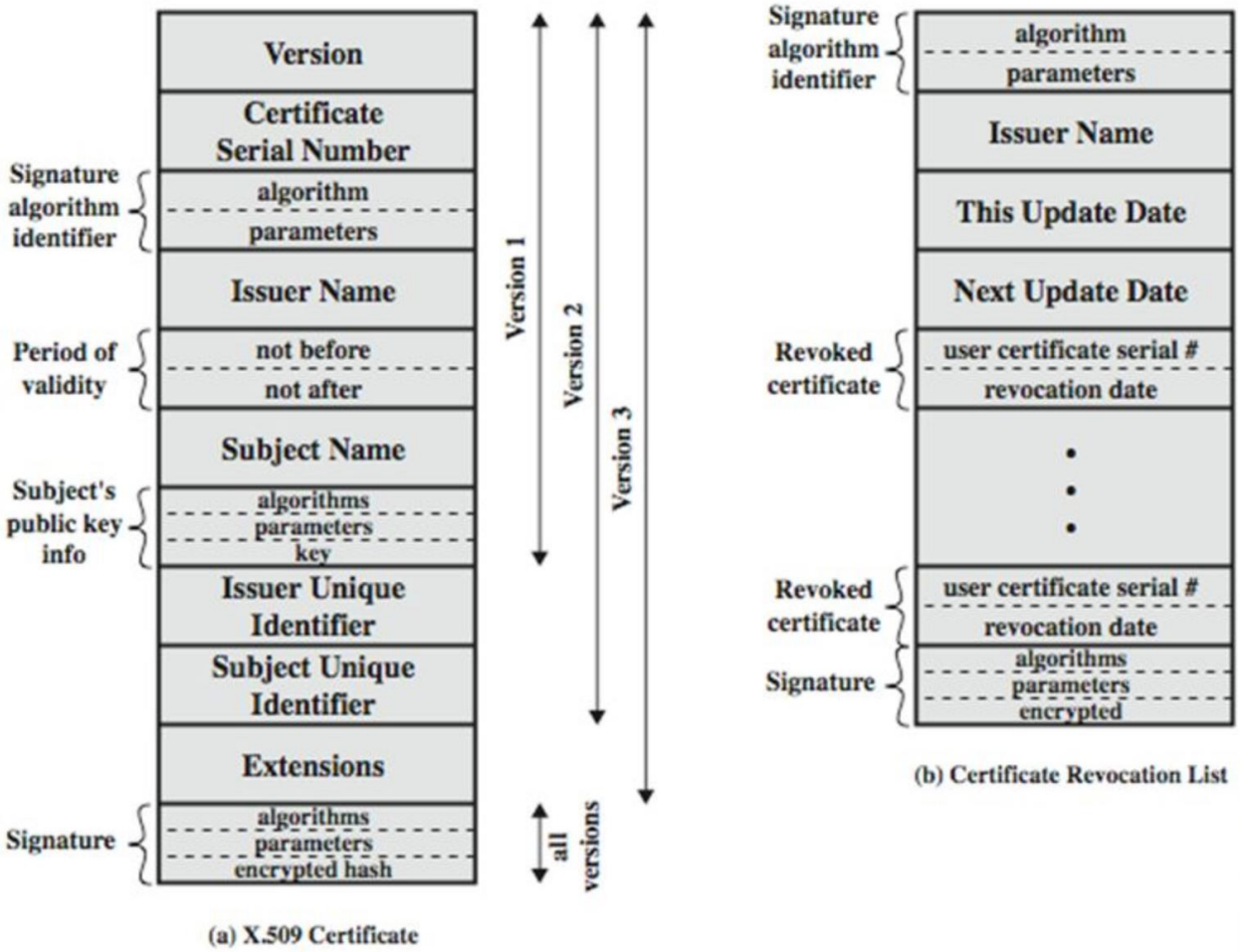

Рис. 2. Формат сертификата открытого ключа и списка аннулированных сертификатов в соответствии со стандартом ITU X.509

Удостоверяющий центр (УЦ) – это специально выделенный участник криптосистемы, которому доверяют все остальные участники, чья цифровая подпись служит гарантией целостности и подлинности ключей и который выполняет следующие функции:

1) сбор сведений об участниках системы, необходимых для выпуска сертификата: имя, почтовый адрес, должность, организация и т.п. (зависит от конкретного приложения);

2) генерация и рассылка (либо помещение в общедоступное хранилище) сертификатов открытых ключей;

3) уничтожение сертификатов с истекшим сроком годности;

4) обновление сертификатов;

5) аннулирование сертификатов.

Аннулирование сертификата может потребоваться в случаях, когда срок санкционированного использования открытого ключа участника системы прерывается досрочно, ранее, чем это предусмотрено принятым в системе регламентом, например, при компрометации секретного ключа участника криптосистемы, соответствующего данному открытому ключу, при удалении (выбытии) владельца ключа из системы, при смене роли пользователя в системе (перемещении пользователя). Аннулирование сертификата – это чрезвычайное обстоятельство, о котором необходимо оповестить всех участников криптосистемы. Существуют два способа решения этой задачи.

- *Проверка статуса сертификата в режиме реального времени.* Для этого требуется выполнение специального протокола с удостоверяющим центром, который отвечает на вопрос, не был ли запрошенный сертификат аннулирован.

- *Периодическое создание и рассылка списков аннулированных сертификатов* (CRL – Certificate Revocation List). Формат CRL также определен в стандарте ITU X.509 (рис. 2).

Второй способ на практике используется чаще. Однако у него есть один существенный недостаток: между двумя последовательными рассылками списка аннулированных сертификатов всегда существует какой-то временно́й «зазор», т.е. об аннулировании сертификата какого-либо участника все остальные участники узнают не мгновенно, а только по прошествии некоторого времени. Наличие такого разрыва создает угрозу несанкционированного использования аннулированного ключа.

УЦ рассылает всем участникам системы свой открытый ключ, который нужен им для проверки подписи на сертификатах. Считается, что подменить его невозможно в силу трех причин: массовости рассылки, периодического повтора и общедоступности.

Сертификаты всех участников криптосистемы могут либо храниться в специальном общедоступном хранилище – репозитории сертификатов (РС), либо рассылаться по сети. На практике преимущественно используется первый способ, причем физически РС чаще всего реализуется либо с использованием директориального сервиса (директория, к которой открыт доступ на чтение всем участникам), либо веб-сервиса (веб-страница, с которой все участники криптосистемы могут забирать сертификаты).

Преимущество сертификата в том, что два участника системы, доверяющие одному и тому же УЦ, могут не знать и не хранить открытые ключи никаких других участников, а при необходимости обратиться в РС и получить необходимые ключи. Для этого им достаточно знать только открытый ключ УЦ. Это позволяет развёртывать ИОК в криптосистемах со сколь угодно большим и даже неопределенным числом участников, где участники не покрыты сетью непосредственных контактов между собой.

Теперь для того, чтобы узнать открытый ключ любого интересующего его участника, участнику криптосистемы (обозначим его *B*) необходимо однократно приобрести аутентичный открытый ключ УЦ (что технически реализовать не сложно). Далее для установления связи с участником *A* ему необходимо:

1) приобрести сертификат открытого ключа *A* одним из следующих способов: обратившись в РС, непосредственно получив его от УЦ или от участника *A* (зависит от порядка, установленного в системе);

2) выполнить процедуру проверки сертификата, состоящую из следующих действий:

- проверки текущей даты и времени и сравнения с периодом действия сертификата;
- проверки действительности в данный момент времени открытого ключа самого удостоверяющего центра;
- проверки цифровой подписи УЦ на сертификате открытого ключа участника *A*, используя открытый ключ УЦ;
- проверки, не был ли сертификат аннулирован к текущему моменту времени;

3) в случае, если все проверки окончились с положительным результатом, принять открытый ключ, извлеченный из сертификата *A*, как аутентичный ключ.

Далее *B* может использовать открытый ключ участника *A* для выполнения любых необходимых ему криптографических алгоритмов, протоколов или криптосхем. К примеру, участники *A* и *B*, приобретя таким образом открытые ключи друг друга, могут выработать общий секретный ключ по методу Диффи – Хеллмана.

2. Рабочее задание

В лабораторной работе требуется реализовать модели Удостоверяющего центра и по меньшей мере двух обычных участников криптосистемы.

2.1. Реализация модели удостоверяющего центра

Необходимо реализовать модель Удостоверяющего центра, который выпускает сертификаты открытых ключей участников криптосистемы.

В лабораторной работе по желанию можно использовать либо форматы сертификата и CRL согласно стандартам X.509 и RFC 2459 [1], либо применить описанные ниже упрощённые форматы.

Предлагаемый упрощённый формат сертификата:

Серийный номер сертификата (целое число)
Идентификатор алгоритма цифровой подписи УЦ (символьная строка, например, «ГОСТ-512»)
Имя УЦ (символьная строка)
Метка времени начала действия сертификата (дата, время)
Метка времени окончания действия сертификата (дата, время)
Идентификатор алгоритма, для которого предназначен удостоверяемый ключ (символьная строка, например, «ECDH»)
Имя или др. учётные данные участника, чей открытый ключ удостоверяется сертикатом (одна или несколько символьных строк)
Значение удостоверяемого открытого ключа (зависит от криптосистемы, может быть одно или несколько чисел, битовых строк и пр.)
Подпись УЦ

Предлагаемый упрощённый формат CRL:

Идентификатор алгоритма цифровой подписи УЦ (символьная строка, например, «ГОСТ-512»)
Имя УЦ (символьная строка)
Метка времени выпуска текущего CRL (дата, время)
Метка времени выпуска следующего CRL (дата, время)
Список аннулированных сертификатов в формате: Серийный номер сертификата (целое число) Метка времени аннулирования сертификата (дата, время)
Подпись УЦ

Модель Удостоверяющего центра должна удовлетворять следующим требованиям:

- иметь собственную пару ключей цифровой подписи (схему подписи выбрать самостоятельно, предпочтительным выбором является схема ГОСТ 34.10-2018);
- поддерживать репозиторий сертификатов участников, выпущенных Удостоверяющим центром;
- поддерживать список аннулированных сертификатов участников, ранее выпущенных Удостоверяющим центром.

Модель Удостоверяющего центра должна реализовывать следующую функциональность:

- обрабатывать запросы на выпуск сертификата, поступающие от участников криптосистемы, извлекая из запроса идентификатор участника и открытый ключ, проверяя уникальность идентификатора участника и знание им секретного ключа, составляющего пару с открытым ключом, для которого участник просит выпустить сертификат;
- обрабатывать запросы на аннулирование сертификата, поступающие от участников криптосистемы;
- помещать выпущенный по запросу сертификат в репозиторий сертификатов;
- обновлять список аннулированных сертификатов после получения от одного из участников запроса на аннулирование сертификата.

2.2. Реализация модели участника криптосистемы

Модель участника криптосистемы должна удовлетворять следующим требованиям:

- генерировать собственные пары ключей для асимметричных криптосхем (желательно сделать возможной генерацию ключей для разных криптосхем, например, для подписи RSA, протокола Диффи – Хеллмана и пр. в зависимости от запроса пользователя);
- иметь сертификат открытого ключа, выпущенный Удостоверяющим центром.

Модель участника криптосистемы должна реализовывать следующую функциональность:

- запрос сертификатов других участников в репозитории сертификатов;
- проверку сертификатов других участников, включая проверку наличия (отсутствия) сертификата в списке аннулированных сертификатов.

2.3. Реализация модельных протоколов между участниками криптосистемы

В лабораторной работе требуется реализовать криптографические протоколы взаимодействия между моделями Удостоверяющего центра и по меньшей мере двух участников криптосистемы:

- протоколы выдачи Удостоверяющим центром сертификатов открытых ключей Участнику_1 и Участнику_2;
- протокол аннулирования Удостоверяющим центром сертификатов открытых ключей Участника_1 и Участника_2 по запросу от соответствующего участника.

Все остальные решения, касающиеся не упомянутых в описании и рабочем задании аспектов реализации, принимаются студентом самостоятельно.

3. Защита лабораторной работы

1. Изучить принципы функционирования инфраструктуры открытых ключей.

2. Выполнить программную реализацию моделей участников и протоколов между ними в соответствии с рабочим заданием. Должна быть предусмотрена возможность оперативного ввода изменяемых входных данных участников схемы (за исключением долговременных параметров криптосистемы).

3. Подготовить отчёт о лабораторной работе, в котором должны содержаться следующие элементы:

- сведения о языке (-ах) программирования и библиотеках, использованных при реализации протоколов.
- исходный код разработанных программ ***с комментариями***, позволяющими без труда разобраться в коде программы (минимум – для каждого структурного элемента кода на уровне функции/процедуры/объекта/метода должно быть указано его функциональное назначение);
- подготовленные и реализованные примеры успешного и неуспешного выполнения участниками каждого из протоколов.

4. В ходе защиты лабораторной работы продемонстрировать работу программы с использованием заранее подготовленных примеров, а также с измененными по заданию преподавателя исходными данными. Ответить на вопросы преподавателя.

Список источников

1. RFC 2459. Internet X.509 Public Key Infrastructure Certificate and CRL Profile. 1999. URL: https://datatracker.ietf.org/doc/html/rfc2459

Лабораторная работа №2. Реализация модели менеджера паролей

1. Введение

Во многих современных защищенных компьютерных системах основная слабость заключается в пароле пользователя. Очень важно, чтобы пользователи выбирали стойкие пароли (или парольные фразы) для защиты своих учетных записей, но стойкие пароли обычно бывают длинными и громоздкими. Еще более проблематично то, что у пользователя, как правило, есть множество различных сервисов, которые требуют парольной аутентификации, и в результате пользователю приходится запоминать много паролей. Пользователь затрудняется запомнить множество сложных паролей, поэтому на практике часто либо выбирает одинаковые пароли для разных сайтов, либо записывает их на бумажке (в смартфоне, др. местах), что в итоге снижает безопасность.

Один из способов решения этой проблемы – использование менеджеров паролей. Менеджеры паролей позволяют удобно использовать уникальный стойкий пароль для каждого сервиса (сайта), требующего парольной аутентификации. Однако, учитывая конфиденциальность данных, содержащихся в менеджере паролей, необходимо принять меры для безопасного хранения парольной информации и манипулирования ею.

В этой лабораторной работе потребуется реализовать модель защищённого менеджера паролей. При реализации будут использоваться различные криптографические примитивы, в частности, аутентифицированное шифрование, симметричные схемы аутентификации, криптографические хэш-функции.

2. Описание модели менеджера паролей

Как правило, менеджер паролей хранит свою базу паролей на диске, защищая её при помощи выбранного пользователем единственного стойкого мастер-пароля. Во время использования менеджер паролей может временно сохранять открытое представление базы паролей (или её части) в памяти, из которого он предоставляет пароль для каждого желаемого домена. Далее будем предполагать, что менеджер паролей сохраняет базу паролей на диске в хранилище типа «ключевое поле – поле значения» (key-value storage), ключевые поля которого соответствуют доменным именам, а поля значений соответствуют паролям для этих доменов. Пример показан на рис. 1.

Ключ	**Значение**
www.google.com	password
online.vtb.ru	123456
www.amazon.com	6U)qAlOBy%3SZX$o
www.ozon.ru	sekret

Рис. 1. Формат базы паролей, хранимой менеджером паролей на диске

Разумеется, запись этой информации на диск в открытом виде небезопасна: требуется сохранить как конфиденциальность, так и аутентичность данных в хранилище.

Таким образом, следует зашифровать все значения (т.е. пароли для разных доменов), используя схему аутентифицированного шифрования, но нецелесообразно и небезопасно шифровать всю базу паролей в виде одного большого объекта. Если шифровать записи по отдельности, то можно будет дополнять базу паролей новыми записями, вносить в неё дополнения, извлекать отдельные пароли для аутентификации на том или ином сайте, не расшифровывая всю базу паролей.

Указание. В качестве схемы аутентифицированного шифрования рекомендуется использовать ГОСТ Р 34.12-2015 в режиме MGM (рекомендация по стандартизации Р 1323565.1.026–2019).

Обозначим через k_2 ключ для схемы аутентифицированного шифрования. Следует иметь в виду, что если непосредственно зашифровать каждую пару «домен/пароль», то это, очевидно, приведет к утечке некоторой информации о длине паролей. Такая реализация не может считаться безопасной, поэтому необходимо воспрепятствовать тому, чтобы злоумышленник узнал длину пароля. Будем предполагать, что максимальная длина любого пароля составляет 64 байта.

Кроме того, требуется не допустить утечки какой-либо информации о доменах, которые пользователь сохранил в менеджере паролей, сохранив возможность поиска данных, соответствующих определенному домену. Поэтому база паролей должна иметь в качестве своих ключей не сами доменные имена в открытом виде, а их хэш-коды, которые предлагается вычислять с помощью функции HMAC. Тогда, чтобы найти данные, соответствующие доменному имени x, потребуется сначала вычислить $\mathrm{HMAC}(k_1, x)$, где k_1 – секретный ключ для HMAC, и проверить, существует ли такой ключ в базе паролей.

Для получения ключей k_1 и k_2 из имеющегося у пользователя мастер-пароля надлежит использовать следующий способ. С помощью однократного вызова функции PBKDF2 [1, стр. 727] из мастер-пароля вырабатывается главный ключ k. Функция PBKDF2 – это специальная «замедленная» хэш-функция, препятствующая лёгкому перебору паролей по словарю злоумышленником (рис. 2).

A widely used method to construct a slow hash function is called **PBKDF2**, which stands for password based key derivation function version 2. Let F be a PRF defined over $(\mathcal{P}, \mathcal{X}, \mathcal{X})$ where $\mathcal{X} := \{0,1\}^n$. The derived PBKDF, denoted $\mathrm{PBKDF2}_F$, is defined over $(\mathcal{P}, \mathcal{X}, \mathcal{X})$ and works as follows:

$$\mathrm{PBKDF2}_F(pw, salt, d) := \left\{ \begin{array}{l} x_0 \leftarrow F(pw,\ salt) \\ \text{for } i = 1, \ldots, d-1: \\ \qquad x_i \leftarrow F(pw, x_{i-1}) \\ \text{output } y \leftarrow x_0 \oplus x_1 \oplus \cdots \oplus x_{d-1} \in \mathcal{X} \end{array} \right\} \tag{18.6}$$

While (18.6) describes the basic PBKDF2, a simple extension outputs more bits if more are needed. In particular, PKBDF2 can output an element in $\mathcal{X}^b$ for some $1 < b < 2^{32}$ by computing:

$$\mathrm{PBKDF2}_F^{(b)}(pw, salt, d) := \big(\mathrm{PBKDF2}_F(pw, salt_1, d), \ldots, \mathrm{PBKDF2}_F(pw, salt_b, d) \big) \in \mathcal{X}^b \tag{18.7}$$

where all b salts are derived from the provided salt by setting $salt_i \leftarrow salt \parallel \mathrm{bin}(i)$. Here $\mathrm{bin}(i)$ is the binary representation of $i \in \{1, \ldots, b\}$ as a 32-bit string.

Рис. 2. Описание функции PBKDF2

Указания по реализации процедуры выработки главного ключа из мастер-пароля. Важно помнить, что при получении какой-либо новой величины с использованием паролей мы всегда должны использовать случайно сгенерированную соль. Напомним, что смысл использования соли заключается в предотвращении оффлайн-атаки по словарю (см. лекции). Рекомендуемая длина соли

составляет 64 или 128 бит. Главный ключ, полученный из PBKDF2, будет функцией предоставленного ей пароля и соли. Чтобы снова воспроизвести тот же ключ в будущем, например, при загрузке базы паролей с диска, необходимо сохранить соль в открытом виде. Рекомендуется так подобрать параметры функции PBKDF2, чтобы на однократное её вычисление требовалось время порядка 0,5…1 сек. В качестве псевдослучайной функции F внутри PBKDF2 рекомендуется использовать HMAC с криптографической хэш-функцией ГОСТ Р 34.11-2012.

Главный ключ k далее используется в качестве ключа функции HMAC с целью генерации ключа k_1 для хэш-кодов доменных имен и ключа k_2 для шифрования паролей с использованием аутентифицированного шифрования. Чтобы получить эти два ключа, следует вычислить HMAC, взяв два разных произвольных двоичных вектора на входе и используя один и тот же k в качестве ключа HMAC. Таким образом, HMAC здесь используется в качестве псевдослучайной функции.

Указания по реализации функции HMAC. Рекомендуется, чтобы в составе HMAC использовалась криптографическая хэш-функция ГОСТ Р 34.11-2012 с хэш-кодом длиной 256 бит.

Обратите внимание, что менеджеру паролей не разрешается включать мастер-пароль или его хэш-код или любые другие значения, которые могли бы привести к утечке информации о нем, или любой из используемых секретных ключей в базу данных паролей на диске. Кроме того, необходимо иметь в виду, что злоумышленник может получить доступ к Вашему исходному коду – это означает, что в исходном коде тоже не должно встречаться каких-либо значений величин, которые должны быть скрыты от злоумышленника. Следует также уничтожать все переменные, содержащие секретные или одноразовые величины, которые использовались для выполнения каких-либо операций с базой паролей, сразу после окончания этих действий.

Кроме того, необходимо обеспечить защиту менеджера паролей от двух видов атак злоумышленника: *атаки путем перестановки* (swap attack) и *атаки путем отката* (rollback attack).

При *атаке путем перестановки* злоумышленник меняет местами в базе паролей значения, соответствующие разным ключам. Например, злоумышленник может подменить записи для www.google.com и www.ozon.ru. Когда пользователь попытается пройти аутентификацию в www.ozon.ru, пользователь непреднамеренно предоставит свой пароль сайту www.google.com.

При *атаке путем отката* злоумышленник может заменить запись в базе паролей предыдущей версией этой записи. Например, предположим, что злоумышленник получил доступ к базе паролей на диске (рис. 1). Через некоторое время пользователь решил сменить свой пароль для сайта www.google.com на новый, например, google_pwd, что должно привести к изменению соответствующей записи в базе паролей на диске. Однако злоумышленник может заменить эту обновленную запись предыдущей версией записи для www.google.com. Простое использование аутентифицированного шифрования не защищает от этой атаки – вместо этого следует вычислить хэш-код содержимого базы паролей, который сохраняется в отдельном файле. Условно считаем, что он сохраняется на недоступном для злоумышленника носителе, например, на флэш-накопителе пользователя. Всякий раз, когда менеджер паролей загружает базу паролей с диска, необходимо убедиться, что сохраненный и заново рассчитанный хэш-коды совпадают. Таким образом, можно быть уверенным, что содержимое базы паролей не было изменено.

В зависимости от предложенного Вами решения защита от атак путем отката может также обеспечивать и защиту от атак путем перестановки. Однако в настоящей лабораторной работе все равно необходимо реализовать явную защиту от атак путем перестановки. Другими словами, защитные механизмы, которые Вы реализуете, должны работать независимо друг от друга. Даже если хэш-код не предоставлен из надежного

хранилища, менеджер паролей должен быть защищен от злоумышленника, который меняет местами две записи.

3. Рабочее задание

В работе необходимо реализовать модель менеджера паролей, который соответствует описанию, приведенному в п. 2, и удовлетворяет следующим требованиям.

- Для хранения базы паролей должно использоваться хранилище типа «ключевое поле – поле значения» (key-value storage). Базу паролей необходимо хранить в виде файла на диске. Хэш-код базы паролей, используемый для контроля её целостности, сохраняется в отдельном файле, моделируя хранение его на недоступном для злоумышленника носителе.
- Менеджер паролей должен быть защищен мастер-паролем. Недопустимо включать мастер-пароль, криптографические ключи или любые другие величины, которые могли бы привести к утечке информации о них, в базу паролей и в исходный код менеджера паролей. Все переменные, содержащие значения ключей и паролей, а также любые другие величины, которые могли бы привести к утечке информации о ключах и паролях, должны уничтожаться сразу же после завершения операций с ними.
- Криптографические ключи, используемые для защиты базы паролей, каждый раз воспроизводятся из мастер-пароля при помощи функций PBKDF2 и HMAC.
- Должна быть обеспечена защита от атак путем перестановки и отката. Механизмы защиты от этих атак должны быть независимы, в частности, защита от атак путем перестановки должна обеспечиваться, даже если хэш-код базы паролей, используемый для контроля её целостности, не предоставлен.

Все остальные решения, касающиеся не упомянутых в описании и рабочем задании аспектов реализации, принимаются студентом самостоятельно.

4. Защита лабораторной работы

1. Изучить описание программной модели менеджера паролей и рабочее задание.

2. Выполнить программную реализацию модели менеджера паролей.

3. Подготовить и продемонстрировать примеры успешного выполнения менеджером паролей своей функции:

1) вводятся правильный мастер-пароль и адрес сайта, присутствующего в базе паролей, команда «добавить пароль для сайта» – модель добавляет запись в базу паролей;

2) вводятся правильный мастер-пароль и адрес сайта, присутствующего в базе паролей, команда «пройти аутентификацию на сайте» – модель выдаёт пароль для этого сайта;

3) вводятся правильный мастер-пароль и адрес сайта, присутствующего в базе паролей, команда «сменить пароль» – модель вносит изменение в запись базы паролей и записывает новую версию на диск;

4) вводятся правильный мастер-пароль и адрес сайта, присутствующего в базе паролей, команда «удалить пароль» – модель удаляет запись из базу паролей.

В примерах предусмотреть вывод на экран результатов, демонстрирующих получение ключей из мастер-пароля и изменения, вносимые в базу паролей.

4. Подготовить и продемонстрировать любые примеры неудачного выполнения менеджером паролей своих функций:

1) при неправильно введенном мастер-пароле;

2) при вводе адреса сайта, не существующего в базе паролей;

3) при обнаружении попытки противника нарушить целостность базы паролей путем перестановки либо путем отката.

В примерах предусмотреть вывод на экран сообщений об ошибках (останове программы) с указанием причин появления ошибок.

5. Подготовить отчёт о лабораторной работе, в котором должны содержаться следующие элементы:

- сведения о языке (-ах) программирования и библиотеках, использованных при реализации протоколов.
- исходный код разработанных программ ***с комментариями***, позволяющими без труда разобраться в коде программы (минимум – для каждого структурного элемента кода на уровне функции/процедуры/объекта/метода должно быть указано его функциональное назначение);
- подготовленные и реализованные примеры успешного и неуспешного выполнения протоколов участниками (положительные примеры – для всех протоколов, отрицательные – выборочно, для двух протоколов).

6. В ходе защиты лабораторной работы продемонстрировать работу программы с использованием заранее подготовленных примеров, а также с измененными по заданию преподавателя исходными данными. Ответить на вопросы преподавателя.

Список источников

1. Boneh D., Shoup V. A graduate course in applied cryptography. 1135 pp. 2023. URL: https://toc.cryptobook.us/book.pdf

Лабораторная работа №3. Реализация модели системы защищенного распределенного хранения данных

1. Введение

Рассмотрим следующую задачу. Пусть F – некоторый массив данных (или файл), длина этого массива равна N байтов: $|F| = N$, n – количество устройств хранения данных (УХД), на которых этот массив может быть размещен. Необходимо предложить такой метод распределения блоков массива по n УХД, который обеспечивал бы возможность полного восстановления всего массива по информации, содержащейся на любых m УХД, где $1 \leq m \leq n$.

Метод решения этой задачи заключается в следующем. Массив F разбивается на n фрагментов: F_i, $i = \overline{1,n}$, где $|F_i| = \frac{|F|}{m}$ (таким образом, $m|N$) при помощи некоторого алгоритма **Dispersal (F, m, n)**, после чего полученные таким образом фрагменты массива распределяются по УХД. В любой момент времени после распределения фрагментов массива по УХД исходный массив F может быть восстановлен при помощи алгоритма **Recovery** $\left(\left\{\boldsymbol{F_{i_j}}, \boldsymbol{j = \overline{1,m}}, \boldsymbol{1 \leq i_j \leq n}\right\}\right.$**, m, n)**. Заметим, что пока мы рассматриваем эту задачу, не выдвигая требований к обеспечению конфиденциальности массива, т.е. предполагается, что он не содержит информации ограниченного распространения.

Пример. Пусть $|F| = 32$ байта, m=4, n=8. Тогда $|F_i| = 32/4 = 8$ байтов, $i = \overline{1,8}$. Массив распределяется при помощи протокола **Dispersal (F, 4, 8)** на 8 частей, и может быть впоследствии собран, используя любые 4 из них. Например, так: **Recovery ({F1, F3, F4, F7}, 4, 8)**. □

Для решения этой задачи M.O. Rabin в 1989 г. предложил криптосхему, которая получила название IDA (Information Dispersal Algorithm). Она включает в себя алгоритмы распределения блоков массива и восстановления массива из фрагментов.

Алгоритм распределения блоков массива **Dispersal (F, m, n)**.

1. Массив F представляется в виде последовательности байтов, где каждый байт интерпретируется как элемент поля характеристики 2: $F = b_1, b_2, \ldots, b_N$, $b_i \in \mathrm{GF}(2^8)$. Существенными моментами для дальнейшего описания метода здесь являются свойства элементов поля: во-первых, любой ненулевой элемент поля имеет обратный, во-вторых, поле замкнуто относительно операций сложения и умножения.

2. Массив представляется в матричной форме в виде последовательности столбцов: $F = (b_1, \ldots, b_m), (b_{m+1}, \ldots, b_{2m}), \ldots, (b_{N-m+1}, \ldots, b_N)$. Вводятся обозначения: $\overrightarrow{S_i} = \left(b_{(i-1)m+1}, \ldots, b_{im}\right)$, $i = \overline{1, N/m}$. Получается матрица $M = \left[\overrightarrow{S_1}\,\overrightarrow{S_2} \ldots \overrightarrow{S_{N/m}}\right]$ размерности $m \times \frac{N}{m}$.

3. Выбирается матрица A размерности $n \times m$, состоящая из таких строк $\overrightarrow{a_i} = (a_{i1}, \ldots, a_{im})$, $i = \overline{1,n}$, что любые m различных векторов линейно независимы. На эту роль подходит матрица Вандермонда, где $m \leq n$, и все x_i – попарно различные ненулевые элементы поля $\mathrm{GF}(2^8)$:

$$A = \begin{bmatrix} \overrightarrow{a_1} \\ \overrightarrow{a_2} \\ \dots \\ \overrightarrow{a_n} \end{bmatrix}, \qquad A = \begin{bmatrix} 1 & x_1 & x_1^2 & \dots & x_1^{m-1} \\ 1 & x_2 & x_2^2 & \dots & x_2^{m-1} \\ 1 & x_3 & x_3^2 & \dots & x_3^{m-1} \\ \dots & \dots & \dots & \dots & \dots \\ 1 & x_{n-1} & x_{n-1}^2 & \dots & x_{n-1}^{m-1} \\ 1 & x_n & x_n^2 & \dots & x_n^{m-1} \end{bmatrix}.$$

В ней любые m различных строк линейно независимы, так что любая матрица, составленная из m различных строк матрицы Вандермонда, обратима. С подобной матрицей мы уже встречались при рассмотрении пороговых СРС.

4. Фрагменты, размещаемые на разных УХД, вычисляются следующим образом:

$$A \cdot M = \begin{bmatrix} \overrightarrow{a_1} \\ \overrightarrow{a_2} \\ \dots \\ \overrightarrow{a_n} \end{bmatrix} \cdot \begin{bmatrix} \overrightarrow{S_1} & \overrightarrow{S_2} & \dots & \overrightarrow{S_{\frac{N}{m}}} \end{bmatrix} = \begin{bmatrix} \overrightarrow{a_1}\overrightarrow{S_1} & \overrightarrow{a_1}\overrightarrow{S_2} & \dots & \overrightarrow{a_1}\overrightarrow{S_{\frac{N}{m}}} \\ \overrightarrow{a_2}\overrightarrow{S_1} & \overrightarrow{a_2}\overrightarrow{S_2} & \dots & \overrightarrow{a_2}\overrightarrow{S_{\frac{N}{m}}} \\ \dots & \dots & \dots & \dots \\ \overrightarrow{a_n}\overrightarrow{S_1} & \overrightarrow{a_n}\overrightarrow{S_2} & \dots & \overrightarrow{a_n}\overrightarrow{S_{\frac{N}{m}}} \end{bmatrix} = \begin{bmatrix} F_1 \\ F_2 \\ \dots \\ F_n \end{bmatrix}.$$

Каждый элемент произведения матриц равен: $\overrightarrow{a_i} \cdot \overrightarrow{S_k} = a_{i1}b_{(k-1)m+1} + \ldots + a_{im}b_{km} \in \mathrm{GF}(2^8)$. Следовательно, $F_i \in [\mathrm{GF}(2^8)]^{\frac{N}{m}}$.

Алгоритм сборки массива из части фрагментов **Recovery ($\left\{F_{i_j}, j = \overline{1,m}, 1 \le i_j \le n\right\}$, m, n)**.

Пусть даны m фрагментов массива, размещенные на m различных УХД: $\left\{F_{i_j}, j = \overline{1,m}, 1 \le i_j \le n\right\}$. Тогда поскольку

$$\begin{bmatrix} F_{i1} \\ F_{i2} \\ \dots \\ F_{im} \end{bmatrix} = \begin{bmatrix} \overrightarrow{a_{i1}} \\ \overrightarrow{a_{i2}} \\ \dots \\ \overrightarrow{a_{im}} \end{bmatrix} \cdot M = A' \cdot M,$$

и матрица A обратима, матрица M может быть восстановлена из этих фрагментов:

$$\begin{bmatrix} \overrightarrow{a_{i1}} \\ \overrightarrow{a_{i2}} \\ \dots \\ \overrightarrow{a_{im}} \end{bmatrix}^{-1} \times \begin{bmatrix} F_{i1} \\ F_{i2} \\ \dots \\ F_{im} \end{bmatrix} = M.$$

Пример. Пусть, как и в предыдущем примере, $|F| = N = 32$ байта, m=4, n=8. Распределение массива по алгоритму IDA происходит следующим образом:

$$F = b_1, b_2, \dots, b_{32} = (b_1, \dots, b_4), (b_5, \dots, b_8), \dots, (b_{29}, \dots, b_{32}),$$

$$M = \begin{bmatrix} \overrightarrow{S_1} & \overrightarrow{S_2} & \dots & \overrightarrow{S_8} \end{bmatrix} = \begin{bmatrix} b_1 & b_5 & \dots & b_{29} \\ b_2 & b_6 & \dots & b_{30} \\ b_3 & b_7 & \dots & b_{31} \\ b_4 & b_8 & \dots & b_{32} \end{bmatrix},$$

$$A = \begin{bmatrix} \overrightarrow{a_1} \\ \overrightarrow{a_2} \\ \dots \\ \overrightarrow{a_8} \end{bmatrix} = \begin{bmatrix} 1 & x_1 & x_1^2 & x_1^3 \\ 1 & x_2 & x_2^2 & x_2^3 \\ \dots & \dots & \dots & \dots \\ 1 & x_8 & x_8^2 & x_8^3 \end{bmatrix}.$$

как видим, матрица M имеет размерность 4×8, матрица A – 8×4. Следовательно, эти матрицы можно перемножить:

$$A \cdot M = \begin{bmatrix} \overrightarrow{a_1} \\ \overrightarrow{a_2} \\ \dots \\ \overrightarrow{a_8} \end{bmatrix} \cdot \begin{bmatrix} \overrightarrow{S_1} & \overrightarrow{S_2} & \dots & \overrightarrow{S_8} \end{bmatrix} =$$

$$= \begin{bmatrix} \overrightarrow{a_1}\overrightarrow{S_1} = b_1 + x_1 b_2 + x_1^2 b_3 + x_1^3 b_4 & \dots & \overrightarrow{a_1}\overrightarrow{S_8} = b_{29} + x_1 b_{30} + x_1^2 b_{31} + x_1^3 b_{32} \\ \dots\dots\dots & \dots & \dots\dots\dots \\ \overrightarrow{a_8}\overrightarrow{S_1} = b_1 + x_8 b_2 + x_8^2 b_3 + x_8^3 b_4 & \dots & \overrightarrow{a_8}\overrightarrow{S_8} = b_{29} + x_8 b_{30} + x_8^2 b_{31} + x_8^3 b_{32} \end{bmatrix} = \begin{bmatrix} F_1 \\ F_2 \\ \dots \\ F_8 \end{bmatrix}.$$

Таким образом, $|F_i| = 8$ байтов, а $\sum_{i=1}^{8} |F_i| = \frac{n}{m} \cdot N = 64$ байта.

Восстановление массива по четырем выбранным фрагментам $\{F_1, F_3, F_4, F_7\}$ осуществляется следующим образом:

$$\begin{bmatrix} F_1 \\ F_3 \\ F_4 \\ F_7 \end{bmatrix} = \begin{bmatrix} \overrightarrow{a_1}\overrightarrow{S_1} & \overrightarrow{a_1}\overrightarrow{S_2} & \dots & \overrightarrow{a_1}\overrightarrow{S_8} \\ \overrightarrow{a_3}\overrightarrow{S_1} & \overrightarrow{a_3}\overrightarrow{S_2} & \dots & \overrightarrow{a_3}\overrightarrow{S_8} \\ \overrightarrow{a_4}\overrightarrow{S_1} & \overrightarrow{a_4}\overrightarrow{S_2} & \dots & \overrightarrow{a_4}\overrightarrow{S_8} \\ \overrightarrow{a_7}\overrightarrow{S_1} & \overrightarrow{a_7}\overrightarrow{S_2} & \dots & \overrightarrow{a_7}\overrightarrow{S_8} \end{bmatrix} = \begin{bmatrix} \overrightarrow{a_1} \\ \overrightarrow{a_3} \\ \overrightarrow{a_4} \\ \overrightarrow{a_7} \end{bmatrix} \cdot M,$$

т.е.

$$M = \begin{bmatrix} \overrightarrow{a_1} \\ \overrightarrow{a_3} \\ \overrightarrow{a_4} \\ \overrightarrow{a_7} \end{bmatrix}^{-1} \times \begin{bmatrix} F_1 \\ F_3 \\ F_4 \\ F_7 \end{bmatrix}. \square$$

2. Описание системы

Алгоритмы распределения блоков массива и сборки массива из части фрагментов могут быть использованы в системе защищенного распределенного хранения данных**,** обеспечивающей доступность и целостность информационного массива при утрате не более t из n УХД, где $t < n$. Пусть U – владелец массива, GW – шлюз для доступа в облачную среду (сервер облачного сервиса), V_i, V_j – серверы или УХД в облачной среде. Схема включает в себя два протокола и один алгоритм:

- Deposit – протокол, в котором пользователь U контактирует со шлюзом GW, размещает файл F и получает подтверждение этого;
- Dispersal – алгоритм, выполняя который УХД перераспределяют между собой фрагменты помещенного на хранение файла (алгоритм выполняется каждым УХД автономно, но синхронно с остальными УХД);
- Retrieval – протокол, в котором пользователь U контактирует со шлюзом GW и забирает обратно файл F.

Для построения этой криптосхемы необходимо выбрать схему цифровой подписи

(ЦП) $(Gen, Sign, Ver)$.

Указания по реализации схемы ЦП. В качестве схемы ЦП рекомендуется выбрать схему ГОСТ 34.10-2018. При использовании другой схемы ЦП следует обосновать её выбор и невозможность использования схемы ГОСТ 34.10-2018.

Для схемы ЦП генерируются три пары ключей: (sk_U, pk_U) – ключи владельца файла, (sk_{V_i}, pk_{V_i}), $i = \overline{1,n}$ – ключи УХД и (sk_{GW}, pk_{GW}) – ключи шлюза. Сертификаты всех открытых ключей создаются Удостоверяющим центром. Кроме того, выбираются $H(\cdot)$ – односторонняя хэш-функция с трудно обнаруживаемыми коллизиями и A – матрица Вандермонда для схемы IDA. Хэш-функция и матрица считаются общеизвестными.

Указания по реализации криптографической хэш-функции. В качестве хэш-функции рекомендуется выбрать ГОСТ 34.11-2018. При использовании другой хэш-функции следует обосновать её выбор и невозможность использования ГОСТ 34.11-2018.

Наконец, фиксируется $t = n - m$ – предельно допустимый порог утраченных УХД, который задается заранее.

Конструкции протоколов и алгоритмов выглядят следующим образом.

Протокол Deposit:

(1) $U \rightarrow GW$: $F, Sign_{sk_U}(F)$

(2) $GW \rightarrow V_j, \forall j$: $F, Sign_{sk_U}(F)$

(3) $V_j, \forall j \rightarrow V_i, \forall i$: $F, Sign_{sk_U}(F)$

(4) V_j из шага (2) $\rightarrow GW$: $Sign_{sk_{V_j}}(U, F)$

(5) $GW \rightarrow U$: $Sign_{sk_{GW}}(U, F)$.

Алгоритм Dispersal:

Каждый V_i, $i = \overline{1,n}$ выполняет следующие действия:

а) для $\forall j$, $j = \overline{1,n}$ вычисляет

$$A \cdot M = \begin{bmatrix} F_1 \\ F_2 \\ \dots \\ F_n \end{bmatrix},$$

где A – матрица IDA, M – массив F в матричной форме;

б) для $\forall j$, $j = \overline{1,n}$ вычисляет $H(F_j)$;

в) сохраняет на своем УХД совокупность данных

$$\{F_i; \{H(F_j), j = \overline{1,n}\}; Sign_{sk_U}(F)\}.$$

Протокол Retrieval:

(1) $U \rightarrow GW$: $Sign_{sk_U}(F)$

(2) $GW \rightarrow V_j, \forall j$: $Sign_{sk_U}(F)$

(3) $V_j, \forall j \rightarrow GW$: $F_j, \{H(F_i), i = \overline{1,n}\}$

(4) GW выполняет следующие действия:

а) для $\forall j$: устанавливает в качестве $H(F_j)$ то значение, которое принимает

большинство из полученных $H(F_j)$;

б) формирует G – множество индексов тех УХД, от которых получены верные хэш-коды: первоначально множество G устанавливается в $\emptyset$ (пустое множество), затем если присланный j-м УХД $H(F_j)$ совпал с установленным значением, то $G \leftarrow G \cup \{j\}$;

в) вычисляет

$$M = A^{-1} \cdot \begin{bmatrix} F_{i1} \\ F_{i2} \\ \dots \\ F_{im} \end{bmatrix},$$

где M – файл F, записанный в матричной форме;

(5) $GW \rightarrow U$: F.

3. Рабочее задание

Выполнить программную реализацию схемы, обеспечивающей доступность и целостность информационного массива при утрате не более t из n УХД, где $t < n$. Для реализации выбрать поле $\mathrm{GF}(2^8)$.

В интерфейсе приложения должны быть наглядно представлены:

- Исходные данные участников протокола (открытые параметры, ключи и другие секретные данные и т.п.);
- Данные, передаваемые по сети каждой из сторон;
- Проверки, выполняемые каждым из участников.

Процесс взаимодействия между сторонами протокола может быть реализован как с применением сетевых технологий, так и при помощи буферных переменных. Также необходимо выделить каждый из этапов протоколов для того, чтобы его можно было отделить от остальных.

Для реализации Удостоверяющего центра и сертификатов открытых ключей участников следует воспользоваться результатами, полученными при выполнении лабораторной работы №1.

Выбор языка программирования, библиотек и иных инструментов реализации осуществляется студентом самостоятельно.

4. Защита лабораторной работы

1. Изучить описание алгоритмов, протоколов и криптосхемы, приведённое в тексте работы.

2. Выполнить программную реализацию протоколов в соответствии с рабочим заданием. Должна быть предусмотрена возможность оперативного ввода изменяемых входных данных участников протокола (за исключением долговременных параметров криптосистемы).

3. Подготовить отчёт о лабораторной работе, в котором должны содержаться следующие элементы:

- сведения о языке (-ах) программирования и библиотеках, использованных при реализации протоколов.
- исходный код разработанных программ ***с комментариями***, позволяющими без труда разобраться в коде программы (минимум – для каждого структурного элемента

кода на уровне функции/процедуры/объекта/метода должно быть указано его функциональное назначение);

- подготовленные и реализованные примеры успешного и неуспешного выполнения протокола участниками (минимум – по одному положительному и отрицательному примеру).

4. В ходе защиты лабораторной работы продемонстрировать работу программы с использованием заранее подготовленных примеров, а также с измененными по заданию преподавателя исходными данными. Ответить на вопросы преподавателя.

Лабораторная работа №4. Реализация модели защищенного канала передачи данных согласно спецификации TLS 1.3

1. Спецификация TLS 1.3

Спецификация TLS 1.3 включает в себя протокол образования защищенного канала передачи данных и протокол передачи данных по защищенному каналу.

Протокол образования защищенного канала передачи данных. Криптографическое ядро протокола образования защищенного канала передачи данных – это протокол распределения ключей, показанный на рис. 1. Протокол поддерживает как одностороннюю, так и двустороннюю аутентификацию участников. На рисунке показан протокол со взаимной аутентификацией. При односторонней аутентификации сервер проходит аутентификацию для клиента, обратное не требуется.

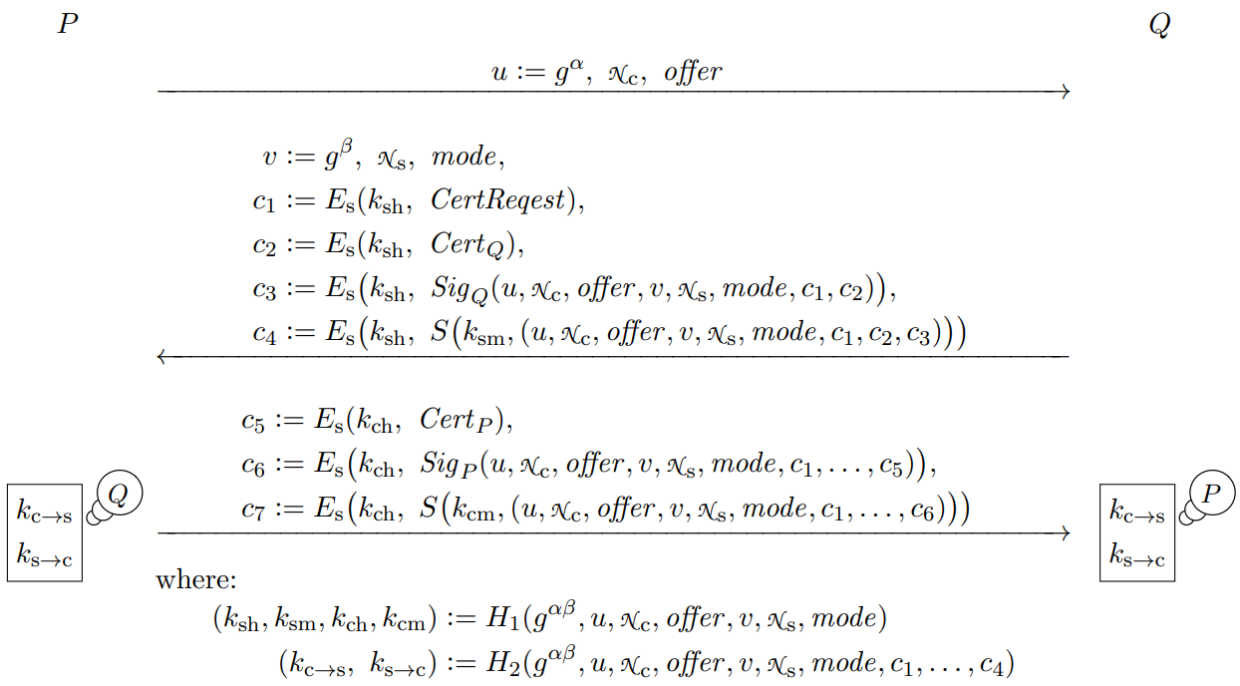

Рис. 1. Протокол распределения ключей, выполняемый при образовании защищенного канала передачи данных

Пусть:

- $\mathbb{G}$ – алгебраическая группа, пригодная для выполнения обмена ключами по методу Диффи – Хеллмана (в TLS 1.3 выбор ограничен несколькими предопределенными алгебраическими группами, включая группы точек эллиптических кривых и подгруппы конечных полей);
- (E_s, D_s) – симметричная схема аутентифицированного шифрования (выбирается из предопределенного списка, например, AES в режиме GCM);
- S – алгоритм генерации кодов аутентификации сообщений (выбирается из предопределенного списка, например, HMAC-SHA256);
- $Sig_P(\cdot), Sig_Q(\cdot)$ – подписи сообщений на секретных ключах участников P и Q соответственно;

- H_1, H_2 – функции выработки производных ключей (обычно HKDF с криптографической хэш-функцией, выбираемой из предопределённого списка, например, SHA256).

Выбор конкретной алгебраической группы и алгоритмов аутентифицированного шифрования, подписи, а также криптографической хэш-функции, используемой в HMAC и HKDF, определяется соглашением участников протокола в первых двух сообщениях (*offer* – клиент отправляет серверу приоритетный список поддерживаемых им наборов криптографических примитивов, *mode* – сервер отвечает клиенту, какой набор примитивов он выбирает – обычно первый из приоритетного списка клиента, который он тоже поддерживает). Все реализации спецификации TLS 1.3 должны, как минимум, поддерживать алгоритм шифрования AES-128 в режиме GCM и криптографическую хэш-функцию SHA256.

Протокол распределения ключей выполняется следующим образом. В первом сообщении клиент *P* пересылает серверу *Q* выбранный им элемент алгебраической группы $u \coloneqq g^{\alpha}$, где g – образующий элемент группы, одноразовое число $\mathcal{N}_c$ и *offer*. На практике клиент может предложить сразу несколько алгебраических групп с их образующими элементами и передать соответствующие числа *u* для каждой из них.

После получения первого сообщения сервер *Q* анализирует *offer*, полученный от клиента *P*. Он проверяет, что алгебраическая группа (группы), которую предпочитает клиент, соответствует группе (группам), с которыми способен работать сервер. После этого сервер выбирает из приоритетного списка клиента первый по порядку набор криптографических примитивов, который он поддерживает (схему аутентифицированного шифрования (E_s, D_s) и криптографическую хэш-функцию, используемую в HMAC и HKDF). Если сервер не способен найти совместимую с клиентом алгебраическую группу и криптографические примитивы, то он может направить специальный запрос клиенту для повторения попытки соединения (подробности здесь не рассматриваются).

После согласования криптографических примитивов сервер отправляет клиенту ответное сообщение, открытая часть которого содержит элемент алгебраической группы $v \coloneqq g^{\beta}$, одноразовое число $\mathcal{N}_s$ и *mode*. Это же сообщение содержит несколько зашифрованных полей:

- c_1 – запрос сертификата *CertRequest*, он отправляется только если сервер требует взаимную аутентификацию от клиента;
- c_2 – сертификат сервера, включающий ключ проверки подписи сервера;
- c_3 – подпись на секретном ключе сервера транскрипции предыдущей части протокола, включая c_1, c_2;
- c_4 – код аутентификации, вычисленный при помощи HMAC, для транскрипции предыдущей части протокола, включая c_1, c_2, c_3.

Ключ k_{sh}, используемый для зашифрования полей сообщения, и ключ k_{sm}, используемый для их аутентификации, вырабатываются из набора данных:

$$g^{\alpha\beta}, u, \mathcal{N}_c, offer, v, \mathcal{N}_s, mode,$$

используя HKDF.

После получения сообщения от сервера клиент отвечает ему сообщением, которое состоит из нескольких зашифрованных полей:

- c_5 – сертификат клиента, включающий ключ проверки подписи клиента (отправляется, если сервер в предыдущем сообщении запросил взаимную аутентификацию);

- c_6 – подпись на секретном ключе клиента транскрипции предыдущей части протокола, включая $c_1, \dots, c_5$;
- c_7 – код аутентификации, вычисленный при помощи HMAC, для транскрипции предыдущей части протокола, включая $c_1, \dots, c_6$.

Ключ k_{sh}, используемый для зашифрования полей сообщения, и ключ k_{sm}, используемый для их аутентификации, вырабатываются из набора данных:

$$g^{\alpha\beta}, u, \mathcal{N}_c, offer, v, \mathcal{N}_s, mode,$$

используя HKDF.

Шифрование полей второго и третьего сообщений протокола необходимо для того, чтобы пассивный противник, перехватывающий сообщения в канале связи, не смог установить идентификаторы участников P и Q из содержания их сертификатов (обеспечивается свойство identity protection).

Клиент после получения сообщения от сервера, а сервер после получения второго сообщения от клиента вырабатывают сеансовые ключи $k_{c\to s}$ и $k_{s\to c}$ из набора данных:

$$g^{\alpha\beta}, u, \mathcal{N}_c, offer, v, \mathcal{N}_s, mode, c_1, c_2, c_3, c_4,$$

используя HKDF.

Протокол передачи данных по защищенному каналу. Ключ $k_{c\to s}$ используется в качестве ключа симметричной схемы аутентифицированного шифрования для защиты сообщений, передаваемых от клиента к северу, а $k_{s\to c}$ – от сервера к клиенту. Сервер может передать свое первое сообщение по защищенному каналу одновременно со вторым сообщением протокола образования защищенного канала передачи данных, а клиент – одновременно с третьим сообщением этого протокола. По этой причине c_5, c_6, c_7 не используются для вычисления ключей $k_{c\to s}$ и $k_{s\to c}$. Разумеется, любое сообщение, переданное сервером клиенту вместе со вторым сообщением протокола образования защищенного канала передачи данных, будет передано неаутентифицированному клиенту. Если требуется взаимная аутентификация, следует дождаться окончания протокола.

Спецификация TLS 1.3 предусматривает возможность обновления ключей защищенного канала. В долгоживущих TLS-соединениях желательно периодически обновлять ключи, чтобы их возможная компрометация не позволила противнику раскрыть открытый текст ранее переданных сообщений. В любой момент после завершения образования защищенного канала P или Q могут запросить обновление ключей, направив другому участнику специальное сообщение *KeyUpdate*, что приводит к выполнению каждым из них следующих вычислений:

$$k'_{s\to c} \coloneqq H(k_{s\to c}), k'_{c\to s} \coloneqq H(k_{c\to s}),$$

где в качестве функции H используется HKDF.

После этого оба участника удаляют прежние ключи $k_{c\to s}$ и $k_{s\to c}$ и шифруют все последующие сообщения, используя $k'_{s\to c}, k'_{c\to s}$. Этот механизм до некоторой степени обеспечивает защиту от чтения назад внутри одного сеанса использования защищенного канала передачи данных.

2. Рабочее задание

2.1. Реализация моделей участников криптосистемы

В лабораторной работе требуется реализовать модели четырёх участников криптосистемы: Удостоверяющего центра, сервера и двух клиентов.

Для реализации Удостоверяющего центра, сертификатов открытых ключей участников и функций проверки сертификатов участниками следует воспользоваться результатами, полученными при выполнении лабораторной работы №1.

2.1.1. Реализация модели Сервера, взаимодействующего с клиентами по защищенным каналам передачи данных, имитирующим TLS 1.3

Модель Сервера должна удовлетворять следующим требованиям:

- иметь сертификаты открытых ключей, выпущенные Удостоверяющим центром, обеспечивающие возможность взаимодействия с Клиентом_1 и Клиентом_2;
- поддерживать криптографические примитивы и протокол распределения ключей, обеспечивающие возможность установления защищенного соединения с Клиентом_1;
- поддерживать криптографические примитивы и протокол распределения ключей, обеспечивающие возможность установления защищенного соединения с Клиентом_2;

Модель Сервера должна реализовывать следующую функциональность:

- проверку сертификатов клиентов, включая проверку наличия (отсутствия) сертификата Сервера в списке аннулированных сертификатов;
- выполнение с Клиентом_1 и Клиентом_2 протоколов установления защищенного соединения, имитирующего TLS 1.3;
- выполнение с Клиентом_1 и Клиентом_2 протоколов передачи сообщений по защищенному каналу, имитирующему TLS 1.3;
- обновление ключей с Клиентом_1 по запросу клиента;
- обновление ключей с Клиентом_2 по инициативе Сервера.

2.1.2. Реализация модели Клиента_1, взаимодействующего с Сервером по защищенному каналу передачи данных, имитирующему TLS 1.3

Модель Клиента_1 должна удовлетворять следующим требованиям:

- иметь сертификат открытого ключа, выпущенного Удостоверяющим центром, обеспечивающий возможность взаимодействия с Сервером;
- поддерживать криптографические примитивы, обеспечивающие возможность установления защищенного соединения с Сервером;
- поддерживать возможность выполнения протокола распределения ключей с Сервером на основе подгруппы конечного поля;
- поддерживать двустороннюю аутентификацию при установлении соединения с Сервером.

Модель Клиента_1 должна реализовывать следующую функциональность:

- проверку сертификата Сервера, включая проверку наличия (отсутствия) сертификата Сервера в списке аннулированных сертификатов;
- выполнение с Сервером протокола установления защищенного соединения, имитирующего TLS 1.3;
- выполнение с Сервером протокола передачи сообщений по защищенному каналу, имитирующему TLS 1.3;
- обновление ключей с Сервером по инициативе клиента.

2.1.3. Реализация модели Клиента_2, взаимодействующего с Сервером по защищенному каналу передачи данных, имитирующему TLS 1.3

Модель Клиента_2 должна удовлетворять следующим требованиям:

- иметь сертификат открытого ключа, выпущенного Удостоверяющим центром, обеспечивающий возможность взаимодействия с Сервером;
- поддерживать криптографические примитивы, обеспечивающие возможность установления защищенного соединения с Сервером;
- поддерживать возможность выполнения протокола распределения ключей с Сервером на основе группы точек эллиптической кривой;
- поддерживать одностороннюю аутентификацию при установлении соединения с Сервером.

Модель Клиента_2 должна реализовывать следующую функциональность:

- проверку сертификата Сервера, включая проверку наличия (отсутствия) сертификата Сервера в списке аннулированных сертификатов;
- выполнение с Сервером протокола установления защищенного соединения, имитирующего TLS 1.3;
- выполнение с Сервером протокола передачи сообщений по защищенному каналу, имитирующему TLS 1.3;
- обновление ключей с Сервером по запросу от сервера.

2.2. Реализация модельных протоколов между участниками криптосистемы

В лабораторной работе требуется реализовать криптографические протоколы взаимодействия между моделями трёх участников криптосистемы: Сервера и двух клиентов, а именно:

- протоколы установления защищенных соединений, имитирующие TLS 1.3, между Сервером и Клиентом_1, Сервером и Клиентом_2;
- протоколы передачи сообщений по защищенным каналам, имитирующим TLS 1.3, между Сервером и Клиентом_1, Сервером и Клиентом_2.

3. Защита лабораторной работы

1. Изучить описание спецификации TLS 1.3.

2. Выполнить программную реализацию моделей участников и протоколов между ними в соответствии с рабочим заданием. Должна быть предусмотрена возможность оперативного ввода изменяемых входных данных участников схемы (за исключением долговременных параметров криптосистемы).

3. Подготовить отчёт о лабораторной работе, в котором должны содержаться следующие элементы:

- сведения о языке (-ах) программирования и библиотеках, использованных при реализации протоколов.
- исходный код разработанных программ ***с комментариями***, позволяющими без труда разобраться в коде программы (минимум – для каждого структурного элемента кода на уровне функции/процедуры/объекта/метода должно быть указано его функциональное назначение);
- подготовленные и реализованные примеры успешного и неуспешного выполнения протоколов участниками (положительные примеры – для всех протоколов, отрицательные – выборочно, для двух протоколов).

4. В ходе защиты лабораторной работы продемонстрировать работу программы с использованием заранее подготовленных примеров, а также с измененными по заданию преподавателя исходными данными. Ответить на вопросы преподавателя.